DSAI 305 XAI Project: Pneumonia Detection using Custom PEPX Architecture and Explainable AI

Ahmed Ihab
ID: 202301122

Abstract

Deep learning models have demonstrated remarkable performance in medical image analysis, yet their "black-box" nature remains a barrier to clinical adoption. This report details the development, implementation, and evaluation of a custom lightweight convolutional neural network utilizing Projection-Expansion-Projection-eXtension (PEPX) modules to detect pneumonia from chest X-rays. The model achieved an overall accuracy of 80% and an Area Under the Curve (AUC) of 0.865. To ensure transparency, Gradient-weighted Class Activation Mapping (Grad-CAM) was employed as an Explainable AI (XAI) technique to visually validate the model's diagnostic reasoning, successfully identifying bounding box pathologies while exposing algorithmic confounders such as medical wiring.

1 Data Explanation

The dataset consists of frontal-view chest radiographs (X-rays) sourced from the RSNA Pneumonia Detection Challenge. Medical images of this nature require robust handling, as identifying regions of consolidation (fluid or infection) is challenging due to the superposition of anatomical structures like ribs and the heart. The dataset includes radiologist-provided bounding boxes for patients diagnosed with pneumonia, serving as the ground truth for both the classification task and the subsequent XAI visual evaluation.

2 Data Preprocessing

Consistent spatial dimensions and normalized pixel distributions are critical for deep learning convergence. The following preprocessing pipeline was applied to the dataset:

- **Channel Conversion:** Raw DICOM/Grayscale images were converted to 3-channel RGB formats to align with standard 2D convolution expectations.
- **Resizing:** All images were uniformly scaled to a resolution of 480×480 pixels to preserve structural fidelity while balancing computational memory constraints.
- **Normalization:** Pixel intensity values were normalized via scaling ($1/255.0$), ensuring the mathematical tensors fed into the network fell within the $[0, 1]$ range.
- **Batching:** The data was structured into continuous generators, enabling sequential extraction of batch tensors during the training and testing phases.

3 Implementation & Architecture

The core architecture is an optimized variant of COVID-Net, fundamentally built upon the **PEPX (Projection-Expansion-Projection-eXtension)** module.

3.1 The PEPX Module

The PEPX design pattern drastically reduces the parameter count while maintaining a high receptive field. It operates in five distinct mathematical stages:

1. **First-Stage Projection:** A 1×1 convolution compresses the input channels.
2. **Expansion:** A subsequent 1×1 convolution expands the dimensionality to prepare for spatial extraction.
3. **Depth-wise Representation:** A 3×3 depth-wise convolution efficiently learns local spatial textures with minimal computational overhead.
4. **Second-Stage Projection:** Dimensionality is compressed back down.
5. **Extension & Skip Connection:** A final 1×1 convolution aligns the channels, followed by an `Add()` residual connection to prevent vanishing gradients.

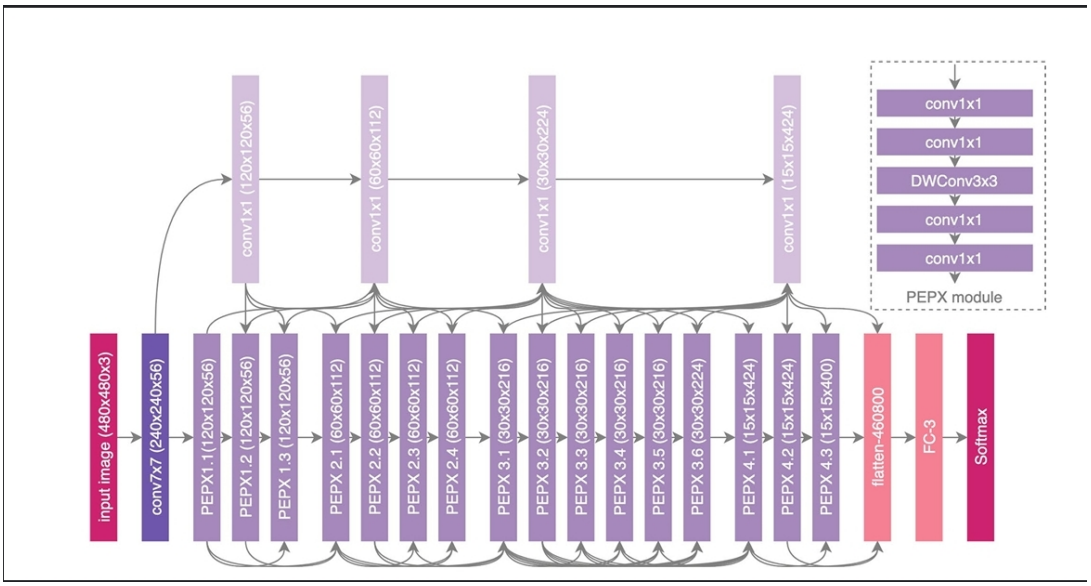

Figure 1: Visual representation of the custom PEPX network architecture.

3.2 Architecture Code Snippet

The exact implementation of the architecture utilized in this study is provided below:

```
from tensorflow.keras.layers import Conv2D, BatchNormalization, Activation,
    DepthwiseConv2D, Add, Input, MaxPooling2D, GlobalAveragePooling2D, Dropout,
    Dense
from tensorflow.keras.models import Model

# The Core Building Block: PEPX Module
def pepx_module(input_tensor, project_filters, expand_filters, out_filters):
    # 1. First-stage Projection (Compress)
    x = Conv2D(project_filters, (1, 1), padding='same')(input_tensor)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # 2. Expansion (Expand)
    x = Conv2D(expand_filters, (1, 1), padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # 3. Depth-wise Representation (Learn Textures)
    x = DepthwiseConv2D((3, 3), padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # 4. Second-stage Projection (Compress)
    x = Conv2D(project_filters, (1, 1), padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # 5. Extension (Final output size)
    x = Conv2D(out_filters, (1, 1), padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # Skip Connection
    if input_tensor.shape[-1] != out_filters:
        shortcut = Conv2D(out_filters, (1, 1), padding='same')(input_tensor)
        shortcut = BatchNormalization()(shortcut)
    else:
        shortcut = input_tensor

    x = Add()([x, shortcut])
    return x

def build_covid_net_binary(input_shape=(480, 480, 3)):
    inputs = Input(shape=input_shape)

    x = Conv2D(64, (7, 7), strides=(2, 2), padding='same')(inputs)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)

    # Stage 1: PEPX Blocks
    x = pepx_module(x, project_filters=32, expand_filters=128, out_filters=256)
    x = pepx_module(x, project_filters=32, expand_filters=128, out_filters=256)
    x = MaxPooling2D(pool_size=(2, 2))(x)

    # Stage 2: PEPX Blocks
    x = pepx_module(x, project_filters=64, expand_filters=256, out_filters=512)
```

```
    x = pepx_module(x, project_filters=64, expand_filters=256, out_filters=512)
    x = pepx_module(x, project_filters=64, expand_filters=256, out_filters=512)
    x = MaxPooling2D(pool_size=(2, 2))(x)

    # Stage 3: PEPX Blocks
    x = pepx_module(x, project_filters=128, expand_filters=512, out_filters
    =1024)
    x = pepx_module(x, project_filters=128, expand_filters=512, out_filters
    =1024)
    x = pepx_module(x, project_filters=128, expand_filters=512, out_filters
    =1024)

    # The Custom Binary Head
    x = GlobalAveragePooling2D()(x)
    x = Dropout(0.5)(x)

    outputs = Dense(1, activation='sigmoid', name='pneumonia_binary_output')(x)
    model = Model(inputs=inputs, outputs=outputs, name='Custom_COVID_Net')
    return model
```

Listing 1: Python implementation of the PEPX Module and Model Architecture.

The network utilizes the Adam optimizer (Learning Rate $= 1e - 4$) and minimizes Binary Crossentropy loss.

4 Performance Measurements

The model was evaluated against an unseen test set of 4,003 images. It demonstrated strong diagnostic capabilities, achieving a macro-average accuracy of 80%.

Class	Precision	Recall	F1-Score	Support
Normal (0)	0.91	0.83	0.87	3101
Pneumonia (1)	0.55	0.71	0.62	902
Accuracy		**0.80** (4003 total)		

Table 1: Final Classification Report

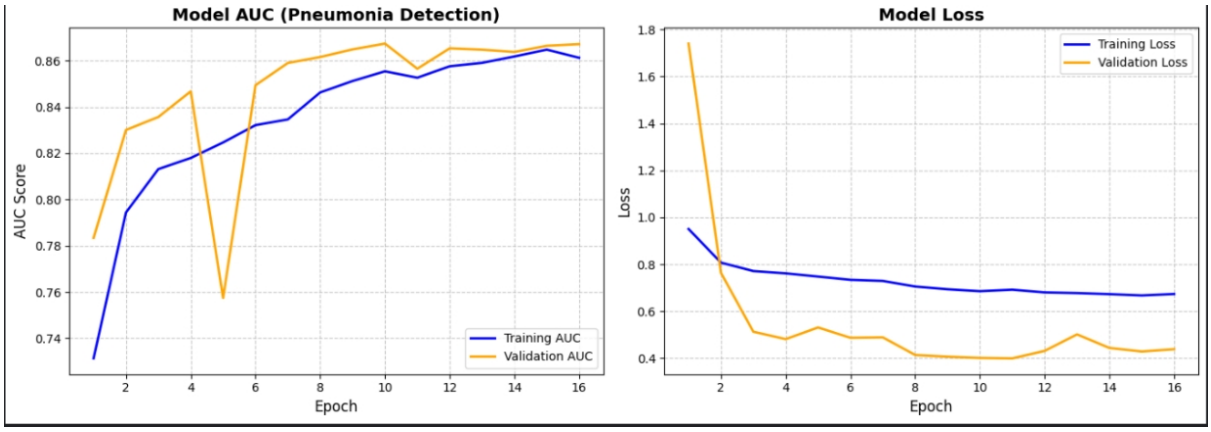

Figure 2: Training History: Model AUC (Left) and Model Loss (Right) across epochs.

As seen in the confusion matrix and ROC curve (Figure 3), the model exhibits a high true negative rate, heavily clearing normal patients successfully. The ROC curve displays an Area Under the Curve (AUC) of **0.865**, indicating excellent discriminative ability between the healthy and pathological classes.

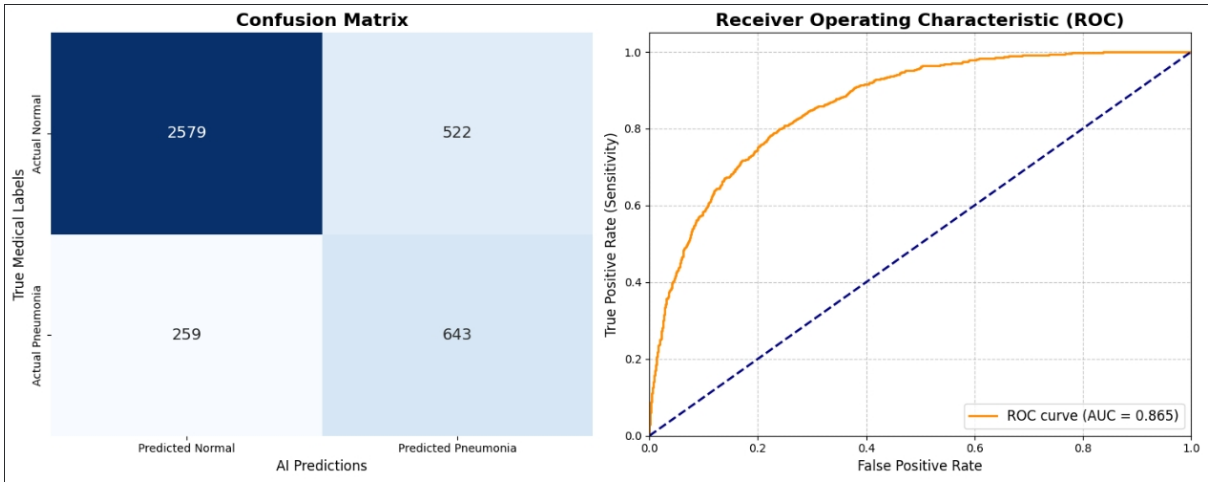

Figure 3: Left: Confusion Matrix. Right: Receiver Operating Characteristic (AUC = 0.865).

5 Explainable AI: Grad-CAM

While quantitative metrics indicate success, clinical models require interpretability. Gradient-weighted Class Activation Mapping (Grad-CAM) was utilized to map the mathematical gradients of the target class back to the final spatial feature map.

5.1 True Positives (Success Cases)

In successful True Positive cases, the Grad-CAM heatmaps cleanly align with the green bounding boxes provided by human radiologists. The AI successfully identifies consolidations (cloudy white opacities) in the lung regions.

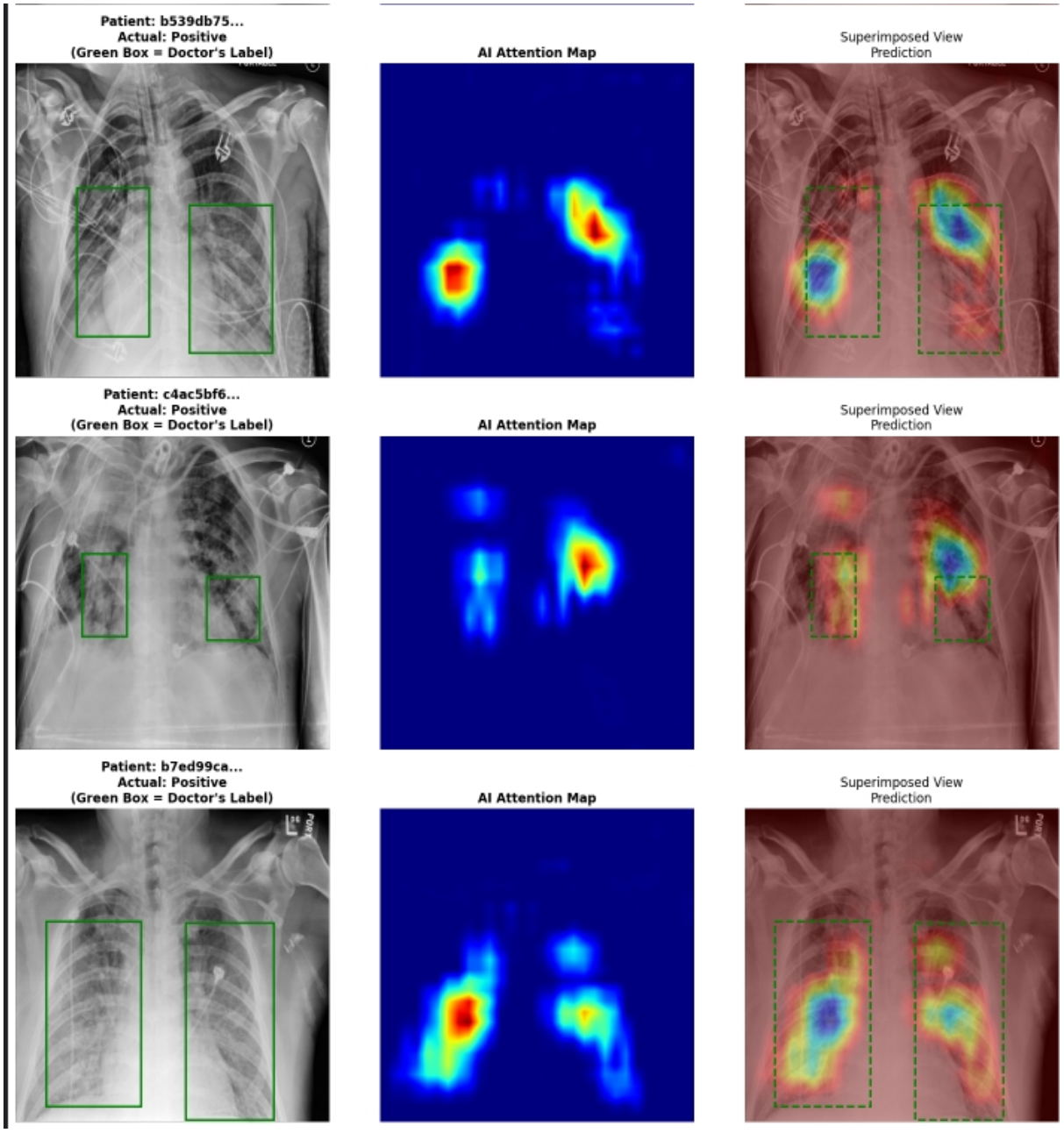

Figure 4: True Positives: Grad-CAM attention aligns directly with actual lung consolidations (Green Boxes).

5.2 False Positives (Confounders & False Alarms)

Analyzing False Positives provides critical engineering feedback. The heatmaps reveal that the model occasionally falls victim to the "Clever Hans" effect. In several instances, the AI utilizes shortcuts—such as highlighting external medical hardware (e.g., ECG wires or chest tubes) running across the patient's chest—as strong indicators of illness, leading to false alarms on healthy patients with hospital monitoring equipment.

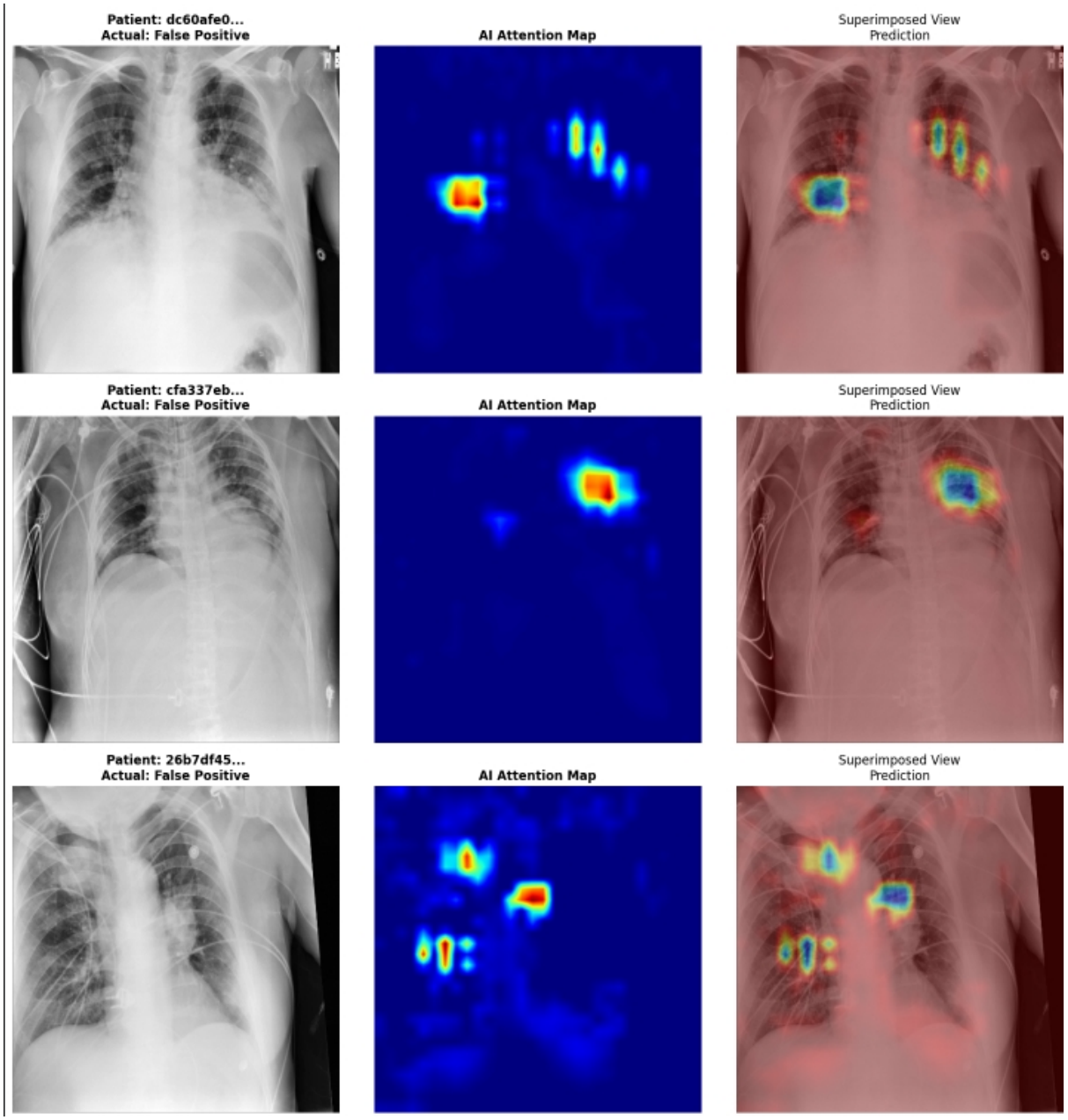

Figure 5: False Positives: Grad-CAM reveals the AI being tricked by confounders, such as medical tubing or distinct bone shadows.

5.3 False Negatives (Missed Diagnoses)

False Negatives occur when the pneumonia consolidation is either highly subtle, hidden behind the heart/diaphragm, or diffuse enough that the localized spatial filters fail to activate strongly. Visualizing these failures ensures the limitations of the current architecture's depth are documented.

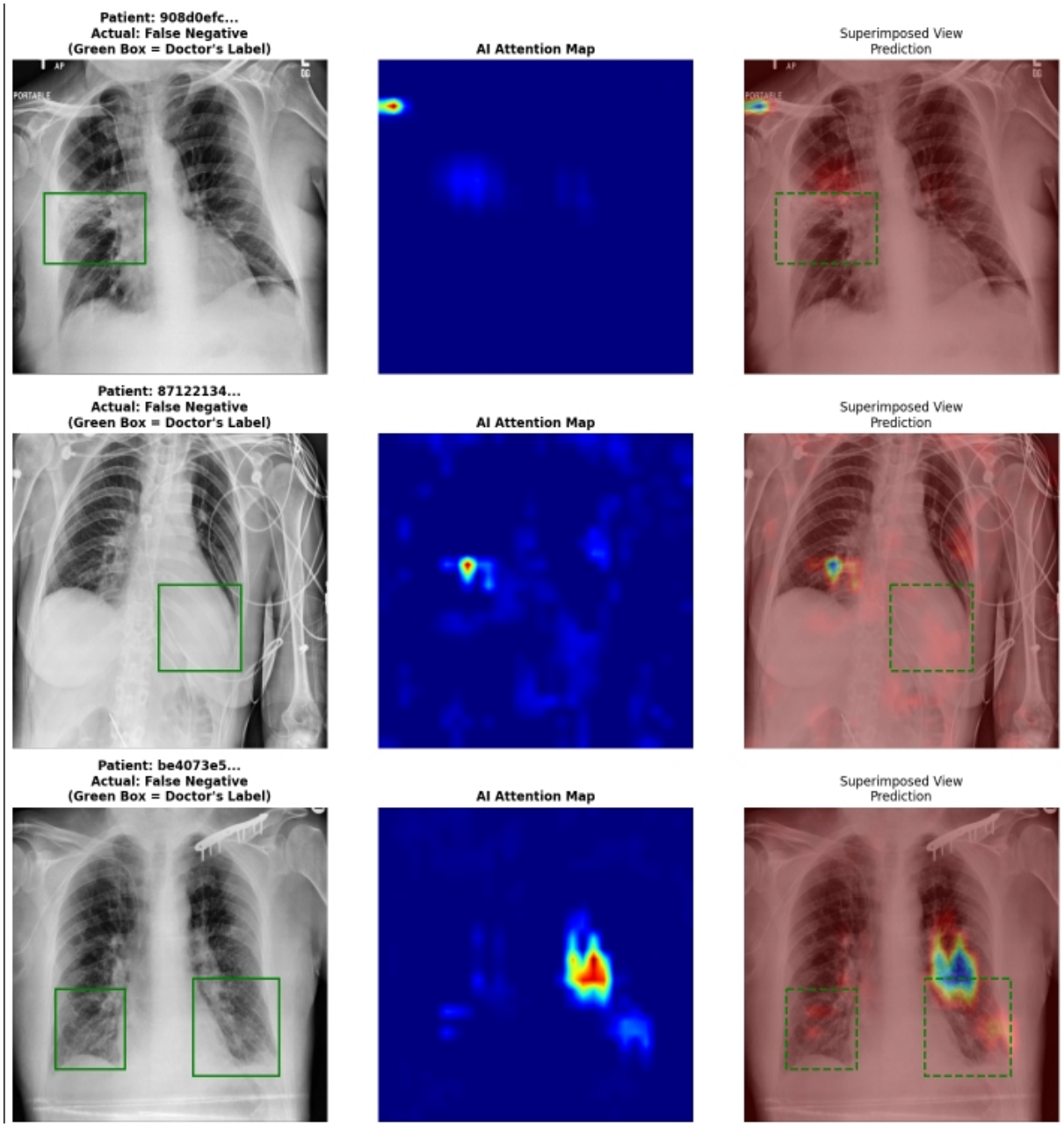

Figure 6: False Negatives: The AI fails to assign high attention to subtle opacities indicated by the true medical labels.

6 Technical Challenges & Resolutions

During the development and extraction of XAI insights, several notable engineering challenges were encountered and successfully mitigated:

- **Cloud Compute vs. Session Detachment:** Managing long training cycles on cloud VM instances initially caused session desynchronization. This was resolved by executing "Save & Run All" commits, successfully bypassing the interactive browser limitations and ensuring state preservation for the generated `.h5` weights.
- **Framework Versioning Anomalies (Keras 3):** Migrating the custom architecture introduced `AttributeError` exceptions when dynamically querying layer output shapes during the Grad-CAM backward pass. This was resolved by implementing a robust type-checking hook (`isinstance(layer, tf.keras.layers.Conv2D)`) rather than relying on tuple length validations.
- **Dimensionality Mismatch in Tensor Processing:** Feeding test arrays back into the loaded model initially triggered a channel mismatch (`expected axis -1 to have value 3, received 1`). Though X-rays are fundamentally grayscale, the network was trained on 3-channel approximations. This was resolved by applying OpenCV color conversion (`cv2.cvtColor(raw_img, cv2.COLOR_GRAY2RGB)`) before tensor expansion.
- **The "1x1 Convolution Trap" in PEPX XAI:** The most significant XAI challenge involved pathological Grad-CAM outputs (resulting in giant, blocky, non-spatial heatmaps). The automated hook algorithm erroneously attached to the final 1×1 dimensionality reduction convolution (`conv2d_35`) rather than a spatial layer. By inspecting the architecture summary and manually overriding the hook to target the final spatial residual block (`add_7`), the code successfully preserved the 30×30 grid mapping, instantly restoring highly localized, clinically relevant heatmaps.

7 Conclusion

The implementation of a highly customized PEPX-based architecture successfully achieved an 86.5% AUC in pneumonia detection. Crucially, the integration of the Grad-CAM XAI factory workflow shifted this project from a "black-box" accuracy metric into a clinically interpretable tool. By analyzing True Positives alongside False Positives and Negatives, the visual evidence confirms the model's fundamental medical efficacy while clearly identifying dataset biases (e.g., medical wire confounders) that must be addressed in future iterations.